

Ask Codex to Run Rethlas

Natural-language proof search as an agent workflow

DSA5210 – Session 4

The Object Is the Codex Request

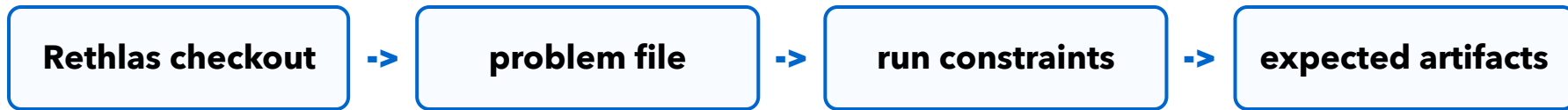
Do not teach this as a list of commands to memorize.

- The instructor gives Codex a mathematical target and a Rethlas checkout.
- Codex studies the local implementation before running anything.
- Codex starts the verifier and generator, then reports artifacts.

The lecture object is the request:

Use Codex to run Rethlas on this problem.

What Codex Receives



Checkout for the demo:

`~/mydoc/hallproblem/Rethlas`

Problem file:

`agents/generation/data/hall_theorem.md`

Codex must keep the run inside the checkout and avoid displaying secrets, auth files, private notes, or token-bearing shell history.

Prompt: Run Rethlas

Use Codex to run Rethlas for this problem.

Checkout:

~/mydoc/hallproblem/Rethlas

Problem file:

agents/generation/data/hall_theorem.md

First inspect the local implementation. Recover the launch contract from README.md, AGENTS.md, run_example.sh, .codex/config.toml, the MCP server, and the verification API.

Then start the required processes, monitor the run, and report the paths of the verified blueprint, verification JSON, logs, and memory.

Do not read or print secrets.

Codex Recovers the Launch Contract

Generation side

- `agents/generation/AGENTS.md`
- `agents/generation/tests/run_example.sh`
- `agents/generation/.codex/config.toml`
- `agents/generation/mcp/server.py`

Verification side

- `agents/verification/AGENTS.md`
- `agents/verification/api/server.py`
- `agents/verification/.codex/config.toml`
- `agents/verification/results/`

Codex should explain what must be running, how the problem path is interpreted, and where outputs will appear.

Two Processes Codex Starts



The verification service exposes `GET /health` and `POST /verify`.

The generation agent reads `data/hall_theorem.md` and calls the verifier when a full proof draft exists.

Codex Checks Before Running

Health

`/health` must answer before generation depends on verification.

Problem path

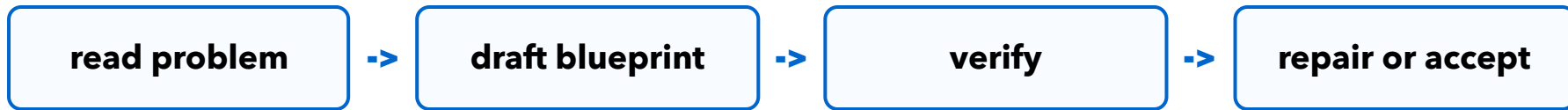
`PROBLEM_FILE` is relative under `data/`.

Secrets

No auth files, keys, private history, or unrelated documents on screen.

These checks are part of the Codex task. The instructor should not manually reconstruct them during the lecture.

The Run Is a Proof Loop



Generator

- decomposes the proof;
- writes logs and memory;
- submits complete drafts.

Verifier

- checks the proof in order;
- records gaps and critical errors;
- returns a structured verdict.

What Codex Reports Back

`blueprint_verified.md`

The proof blueprint accepted by the verifier.

`verification.json`

The structured verifier judgment.

`logs/hall_theorem/`

The human-readable run trajectory.

`memory/hall_theorem/`

Durable proof-search state.

Hall Theorem Demo Files

```
~/mydoc/hallproblem/Rethlas
```

```
agents/generation/data/hall_theorem.md
```

```
agents/generation/results/hall_theorem/blueprint_verified.md  
agents/generation/results/hall_theorem/verification.json
```

```
agents/generation/logs/hall_theorem/hall_theorem.md  
agents/generation/memory/hall_theorem/
```

These files are enough for a reliable classroom demo even if a fresh agent run is slow.

What the Recording Shows

Input

The Hall theorem problem file in `data/`.

Operation

Codex starts and monitors Rethlas.

Output

Verified blueprint, verifier JSON, logs, and memory.

The recording should make the agent workflow visible: Codex studies the checkout, starts the roles, and explains the artifacts.

The Boundary of the Claim

Rethlas produces a verified natural-language proof blueprint.

- The verifier is another Codex agent.
- The output is structured prose plus JSON.
- The process records logs and memory.

This is not a Lean kernel certificate.

- The lecture should not overstate the verification status.
- The result is a natural-language proof accepted by the Rethlas verifier.

Implementation Question

Why does the same Codex executable become a generator in one folder and a verifier in another?